Help! I've Fallen into sPHENIX Analysis and I Can't Get Up!

Valerie Wolfe

February 10th, 2026

Goals of this Talk !!

1. Teach new students the basics of Fun4All
2. Give experienced collaborators a new tool for explaining Fun4All
3. Provide a general introduction to KFParticle

Disclaimer !!

This talk is not intended to teach highly technical computer skills.

I will not teach you how Fun4All works—I will teach you how to *use* it.

Please use the information presented here in conjunction with other, more correct sources.

Recommended sources:

Talk by Chris Pinkenburg *The sPHENIX wiki* *The sPHENIX GitHub*

(Chris Pinkenburg, please don't look at this talk. You will be disappointed in me.)

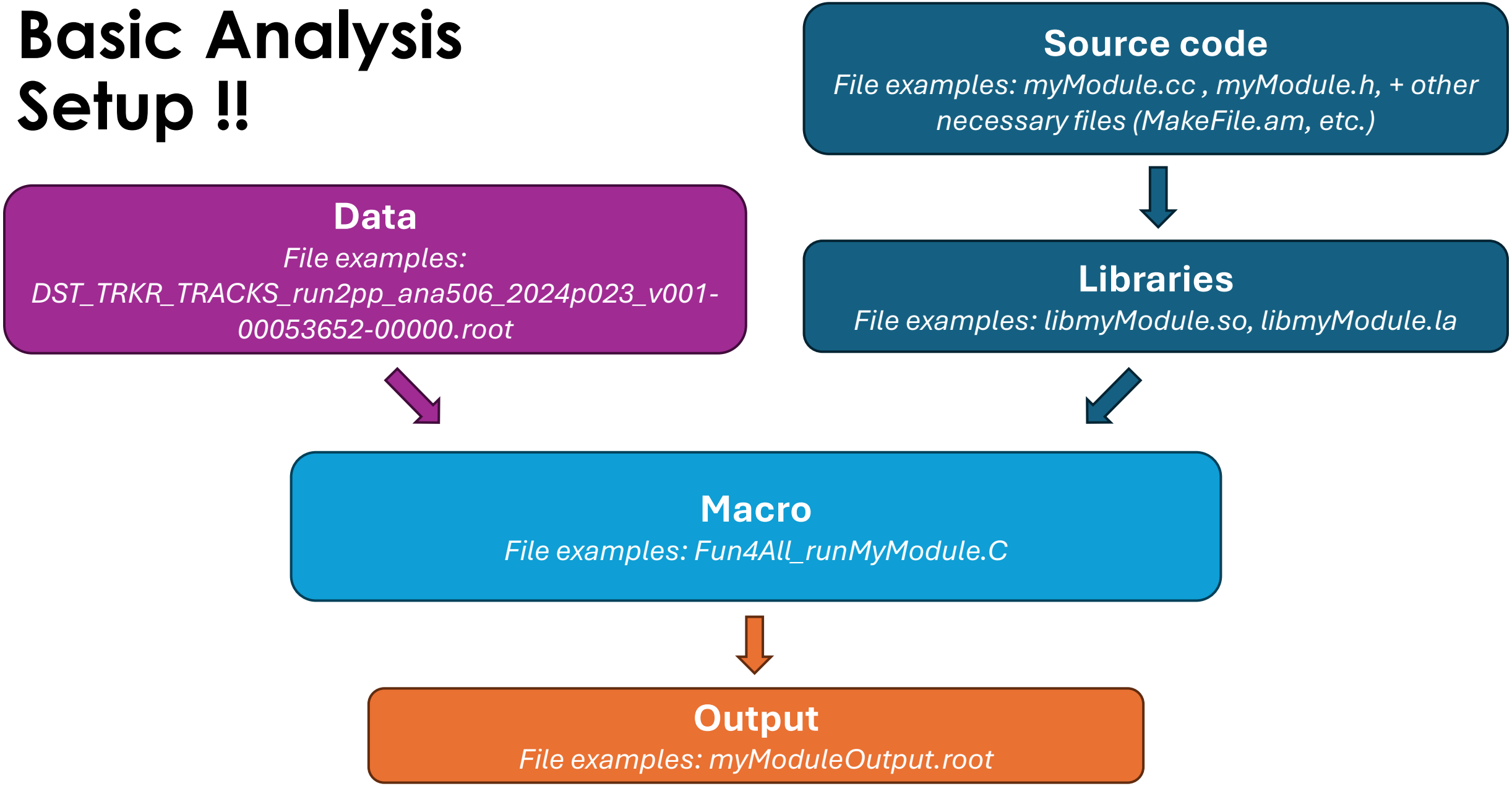
Background !!

Computers, working environment, and analysis structure

Assumptions !!

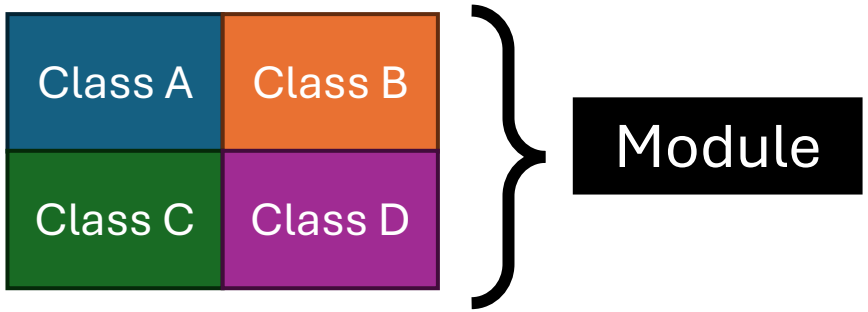
- You know what a subsystem is/the basic structure of sPHENIX
- You know what “macro” means
- You have some basic coding skills (including how to use ROOT)

Basic Analysis Setup !!



Source Code ??

- Code that defines classes to be called in a macro
  - You do not directly run source code!
- Usually grouped into “modules”
  - Example: the sPHENIX KFParticle module is made up of many classes
    - KFParticle_tools, KFPartcile_container, etc.
- Additional files needed: Makefile.am and autogen.sh



Libraries ??

- Compiled code that the computer uses to “do stuff”
  - “Compiled” meaning the computer can directly read and use it
- “Local” (also called “private”) libraries:
  - Code you compile yourself
  - Can refer to entirely custom modules, as well as local versions of existing sPHENIX modules
- “Global” libraries:
  - Pre-compiled, common to everyone in sPHENIX

Ana Build ??

- A stable version of sPHENIX's code base, which is compiled into global libraries
- Builds are indexed with a number: ana517, ana518, etc.
- You should keep track of what build you use in your analysis

New Build ??

- The newest build of the sPHENIX codebase changes daily
- Changes are made LOCALLY (i.e. collaborators make private libraries), then are pushed to our github
- Once approved, these changes are merged with the sPHENIX code base as part of the “new” build
- The new build should be used for development only
  - Research work should use stable ana builds

Working Environment Setup !!

- When you log into SDCC, you should source two setup scripts, in this order:
  1. `sphenix_setup.sh`
  2. `setup_local.sh`
- Ideally, this is done with one script that automatically runs when you log in (i.e. .bashrc)
- These scripts do lots of things, but in particular they tell the computer…
  - where to look for the global libraries (AKA the ana build)
  - where your local libraries are located
  - to use global libraries only if there is no local version for a given module

sphenix_setup.sh

Function: Sets up working environment so you load the libraries belonging to a specified ana (or the new) build

Practical meaning: When creating an object defined in the sPHENIX software, Fun4All will use the version belonging to the ana build you specify

Code in .bashrc file:

```
source /opt/sphenix/core/bin/sphenix_setup.sh -n new
```

Note: You can load ana builds by substituting "new" with any of the supported builds (i.e. "ana509"). New just loads the newest ana build.

Tip: See current supported builds by running

```
ls /cvmfs/sphenix.sdcc.bnl.gov/alma9.2-gcc-14.2.0/release/release_ana/
```

setup_local.sh

Function: Sets up your environment so you load your local libraries before (or, instead of) the global version of that same class.

Practical meaning: Imagine you have compiled a local version of KFParticle. If you create a KFParticle object in a macro, the computer will use your local version of the KFParticle library INSTEAD of the ana build version to create the object.

Code in .bashrc file:

```
export MYINSTALL=/sphenix/user/USERNAME/install  # Set USERNAME to your SDCC username
export LD_LIBRARY_PATH=$MYINSTALL/lib:$LD_LIBRARY_PATH
export ROOT_INCLUDE_PATH=$MYINSTALL/include:$ROOT_INCLUDE_PATH
source /opt/sphenix/core/bin/setup_local.sh $MYINSTALL
```

Note: Not running this script ensures that you only use global libraries.

Misc. Vocab !!

DST

- “Data storage tape”
- The type of data files you will most often use when running Fun4All
- Extension is .root
- Loaded at the macro level
- Example (loading a DST_CALOFITTING file):

```
auto CALOF_in = new Fun4AllDstInputManager("CalofInputManager");
CALOF_in->fileopen(calofDST);
se->registerInputManager(CALOF_in);
```

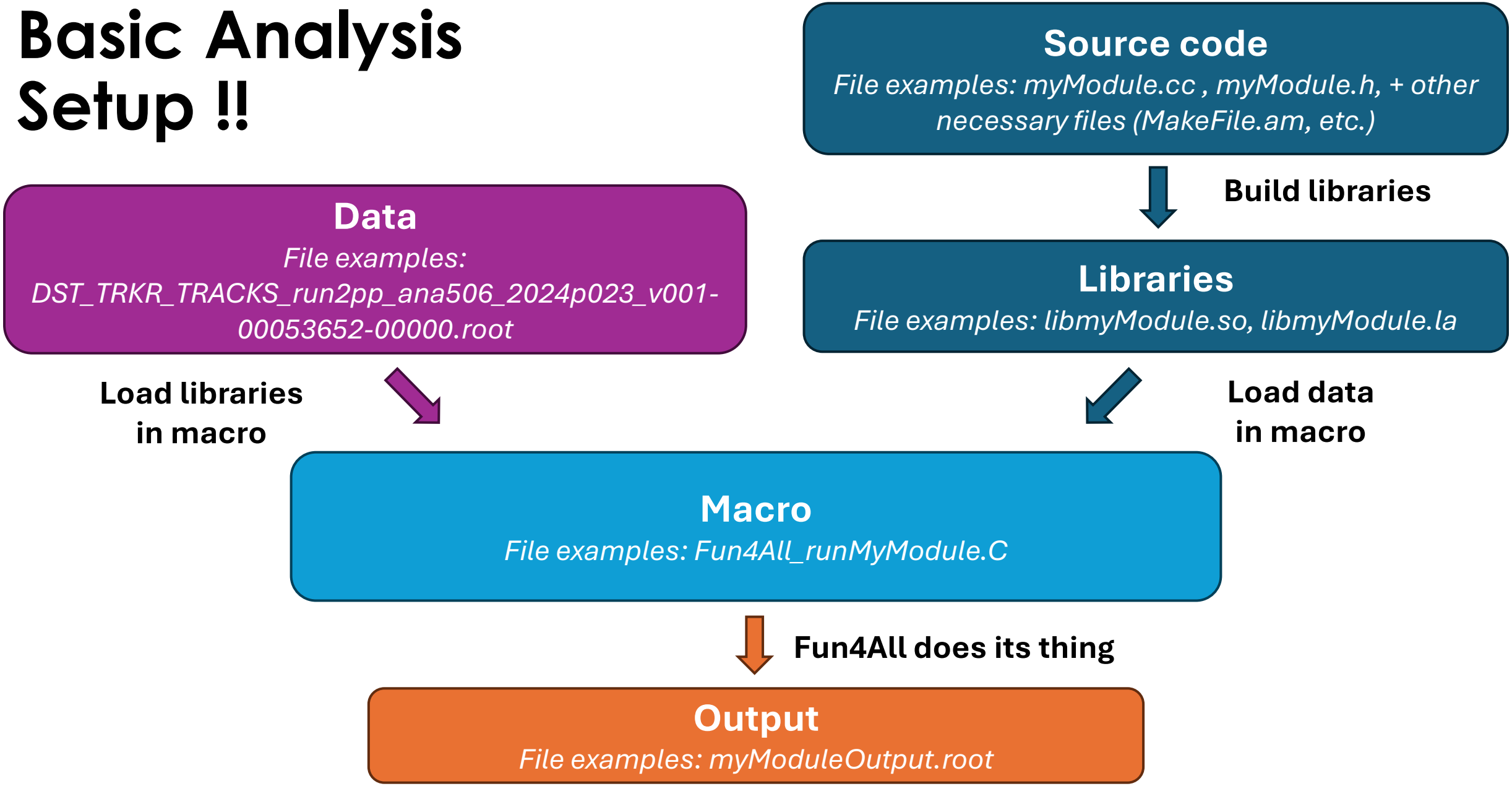
Node Tree

- All the data types available to an individual Fun4All server
- If you are loading a “node” then you are loading a specific data structure
- You do this at the source code level, not at the macro-level
- Example (loading an SVTXTrackMap):

```
SvtxTrackMap *m_trackMap = nullptr;
m_trackMap = findNode::getClass<SvtxTrackMap>(topNode, m_trackMapName);
```

Note: The node tree (all the data types available to Fun4All when your macro ran) should print out in the terminal when you run a macro that uses Fun4All

Basic Analysis Setup !!

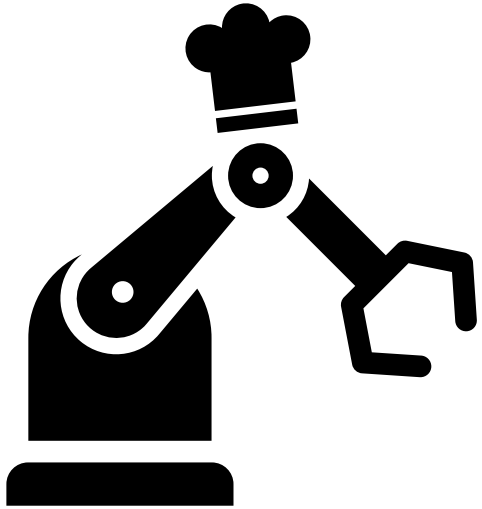


Fun4All !!

What is Fun4All ??

- In the least technical terms possible:

Fun4All is a robot that cooks a delicious meal.



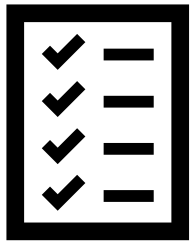
WAIT

I know that sounds ridiculous

Just walk with me here, I promise this will help some of you

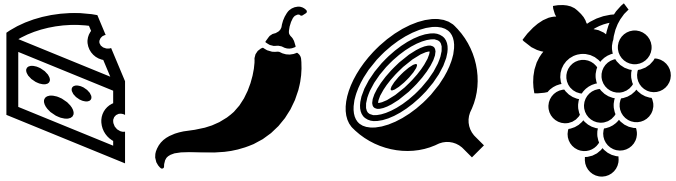
THE RECIPE

AKA the macro



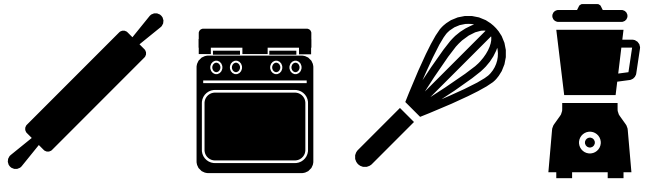
INGREDIENTS

AKA the data



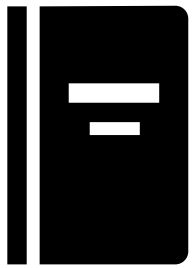
EQUIPMENT

AKA modules



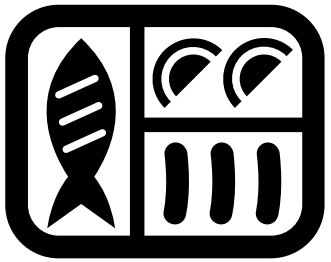
INSTRUCTIONS

AKA libraries and
header files



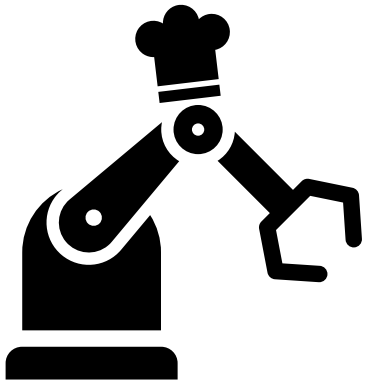
DELICIOUS MEAL

AKA the output
(TTrees, histograms, etc.)



THE ROBOT

AKA Fun4All



CHEF FUN4ALL

The Macro

Give the robot instructions for the tools by including header files and loading libraries.

Turn the robot on by creating a Fun4All server.

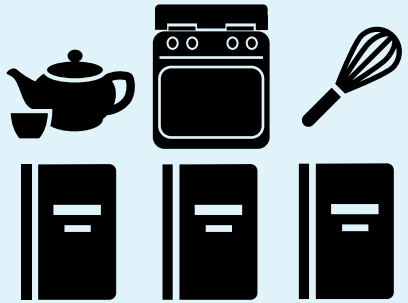
Give the robot necessary ingredients by registering DSTs to the server.

Give the robot necessary tools+recipes by registering modules and setting selections.

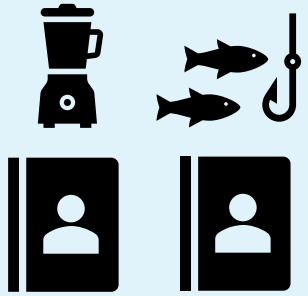
Run the macro with ROOT to create your delicious meal.

SDCC

sPHENIX data



The ana build



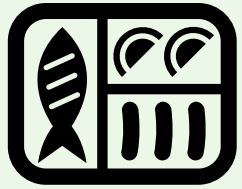
Custom modules

Node Tree

Include Statements

Server

Output



User

A Few Notes !!

- Much like a regular recipe, order matters
  - Don’t tell the robot to scramble the eggs before you’ve cracked them
  - Don’t tell TrackFittingQA to run before you’ve loaded in tracks

Alternative Explanation: Code Snippets !!

- Test module built to look at 5x5 cells in the EMCal
- Found at: `/gpfs/mnt/gpfs02/sphenix/user/vwolfe/Tutorials/TestModule`
- *Note: this module does not create a file as output*

```
#include <caloreco/RawClusterBuilderTemplate.h>
#include <clustershape/ClusterShape.h>
gSystem->Load("libClusterShape.so");
```

Give the robot instructions for the tools by including header files and loading your private libraries.

```
auto se = Fun4AllServer::instance();
```

Turn the robot on by creating a Fun4All server in your macro.

```
auto CALOF_in = new Fun4AllDstInputManager("CalofInputManager");
CALOF_in->fileopen(calofDST);
se->registerInputManager(CALOF_in);
```

Give the robot necessary ingredients by registering DSTs to the server.

```
// Clusters
Rawclus_buildTemplate *clus_build = new Rawclus_buildTemplate("EmcRawclus_buildTemplate");
clus_build->Detector("CEMC");
clus_build->set_UseDetailedGeometry(true);
se->registerSubsystem(clus_build);
// My module
ClusterShape *clutersshape = new ClusterShape("cluster_shape_test");
se->registerSubsystem(clutersshape);
```

Give the robot necessary tools+recipes by registering modules and setting selections.

```
se->run(nEvents);
se->End();
```

Tell the robot how many events to run over. Shut off the server. Enjoy.

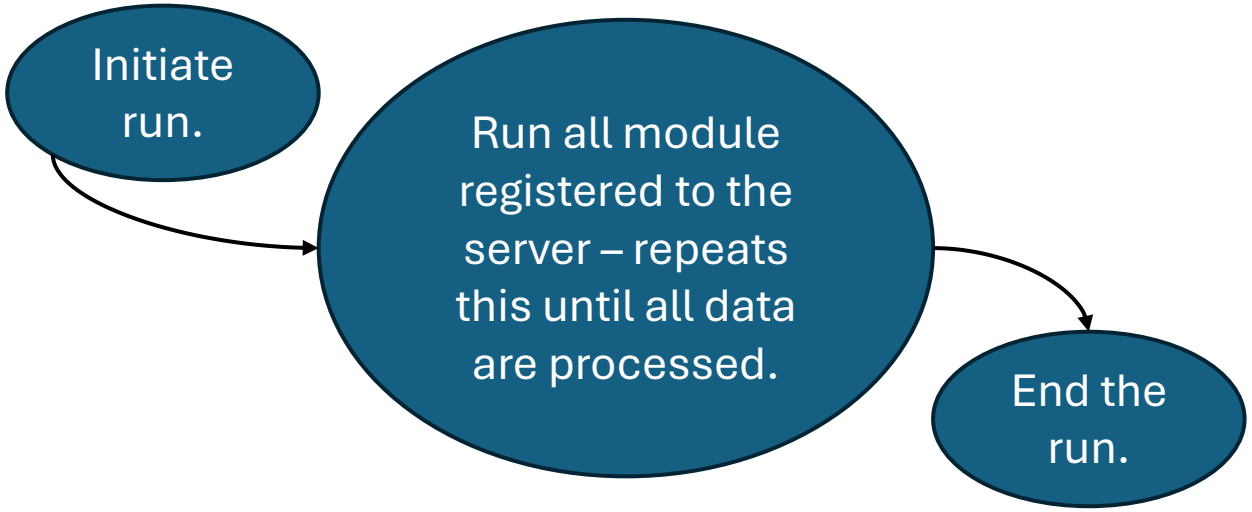
Main function

From Fun4All's Perspective…

Fun4All has a few pre-determined functions, including:

```
// Fun4All functions
int InitRun(PHCompositeNode *topNode) override;
int process_event(PHCompositeNode *topNode) override;
int End(PHCompositeNode *topNode) override;
```

Fun4All calls these functions whenever you tell Fun4All to run.



**There are other functions – Please look at the subsysreco documentation for more info!!*

Module Structure !!

To use Fun4All, a module must tell Fun4All what it should do when running these functions:

```
// Fun4All functions
int InitRun(PHCompositeNode *topNode) override;
int process_event(PHCompositeNode *topNode) override;
int End(PHCompositeNode *topNode) override;
```

This is what the `override` is for – that bit of code tells Fun4All that it should run your module's version of `InitRun`, `process_event`, and `End`, rather than the default versions!!

How ??

This is possible because sPHENIX modules (those that interface directly with Fun4All) all inherit from the “SubsyReco” class:

```
class PHCompositeNode;
class ClusterShape : public SubsysReco
```

Aaaand that’s where this talk runs out of technical details about Fun4All.
Please see recommended resources for more information.

Tools Available to You !!

- CreateSubsysRecoModule
  - Create a Fun4All shell!
- MyOwnTTree
  - Learn how to use Fun4All to make a TTree as output


KFParticle !!

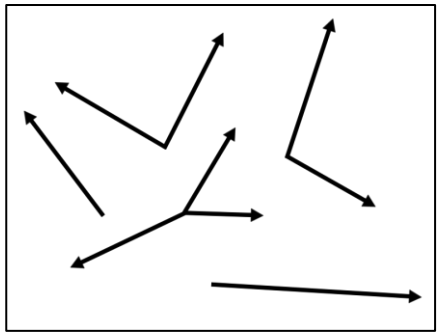
KFParticle (KFP) ??

- Analysis package used in sPHENIX to reconstruct mass resonances
  - One of the tools we can give Fun4All to cook a delicious meal
- **Input**: tracks and your decay of choice
- **Output**: mother and corresponding daughter candidate info in a Ttree
- (KFParticle has some other options for output, please see Cameron's talk for more info)

Decay

$$Y \rightarrow e^+ + e^-$$

Track map



KFParticle

Optional Tree Branches !!

dE/dx

- DST_TRKR_CLUSTER
- `get_dEdx_info()`

Primary vertex info

- DST_CALOFITTING for MBD or DST_TRKR_CLUSTER for SVTX
- `getAllPVInfo()`

Trigger info

- DST_CALOFITTING
- `getTriggerInfo()`

EMCal cluster information

- DST_CALOFITTING
- Additional code in macro (next slide)

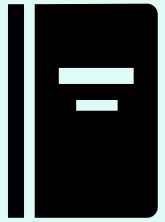
Tracking detector info

- DST_TRKR_CLUSTER
- `getDetectorInfo()`, or
- `GetDetailedTracking()`

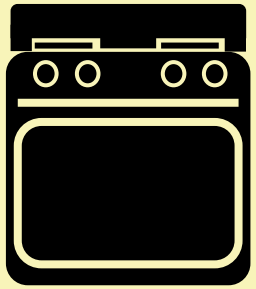
Helpful documentation here.

Chef Fun4All Analogy !!

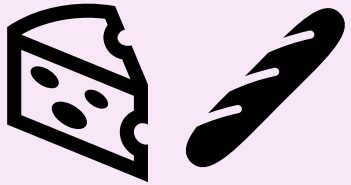
KFParticle libraries

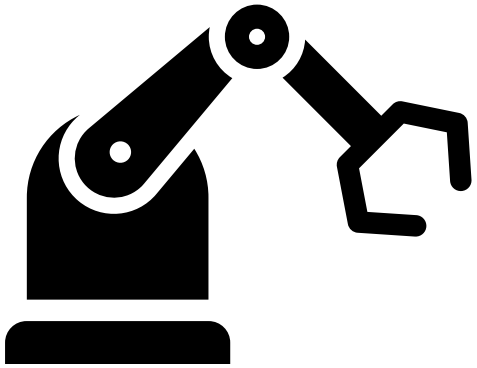


KFParticle module



Track map, optional data types





.root file containing a TTree with any additional data selected by the user (vertex info, etc.)

Loading a Track Map !!

- KFP needs an SvtxTrack map (the output of ACTS)
- Easiest option: DST_TRKR_TRACKS files
- Other options:
  - DST_TRKR_SEED or DST_TRKR_CLUSTER files
  - Important—you must create SvtxTracks before running KFP if using these DSTs
    - Example using ACTS here
- Note: for dE/dx information, you must load in DST_TRKR_CLUSTER files
  - You don’t need to run any tracking if you load in DST_TRKR_TRACKS as well

Loading EMCal Clusters !!

- Need to build clusters based on either MBD or SVTX vertex:

```
RawClusterBuilderTemplate *CB = new RawClusterBuilderTemplate("clus_bld");
CB->Detector("CEMC");
CB->set_UseDetailedGeometry(true);
CB->set_threshold_energy(0.070);
std::string emc_prof = getenv("CALIBRATIONROOT");
emc_prof += "/EmcProfile/CEMCprof_Thresh30MeV.root";
CB->LoadProfile(emc_prof);
CB->set_UseTowerInfo(1);
se->registerSubsystem(CB);
```

Track-Calo Matching !!

- Must create and register the projection:

```
auto projection = new PHActsTrackProjection("CaloProjection");
float new_cemc_rad = 100.70;
projection->setLayerRadius(SvtxTrack::CEMC, new_cemc_rad);
se->registerSubsystem(projection);
```

- Must tell your KFP object that you want the calorimetry info:

```
kfparticle->getCaloInfo(true);
```

There are many matching selections you can set. See track-calo matching software here.

Example Code !!

- Path: `/sphenix/user/vwolfe/Tutorials/PhotonConversion_Tutorial`
- Contains macro, DSTs, and a readme.txt
- No source code – uses the standard KFParticle ana build!

Acknowledgments !!

- Thank you to Cameron Dean, Tristan Protzman, Jaebeom Park, and Rosi Reed!

- Shoutout to Charles Hughes for consulting on slideshow design
  - Me: hey charles do you think all words in a title should be capital?
  - Charles: yes, and also bolded, with two exclamation points